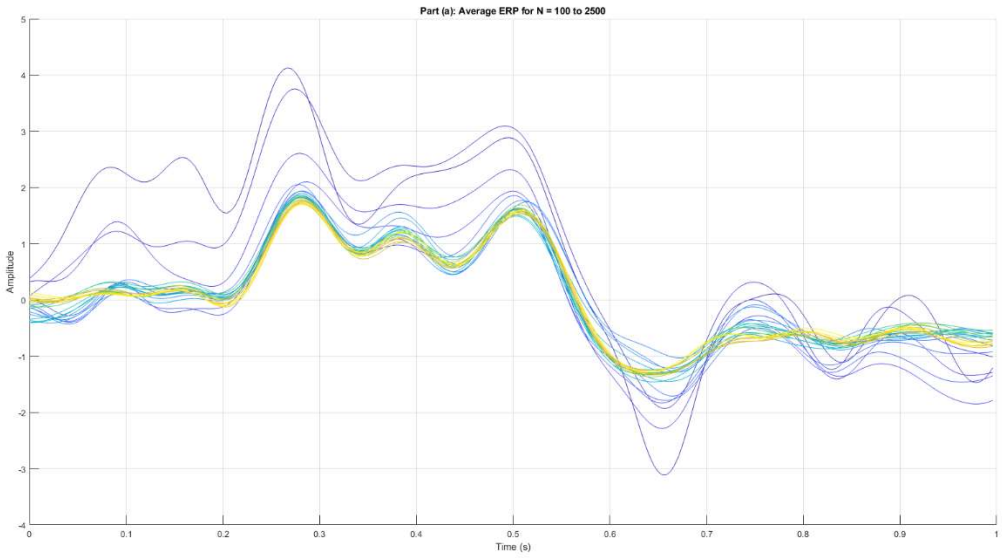
Section 1

Part1

Using the MATLAB script below, we plot the following results:

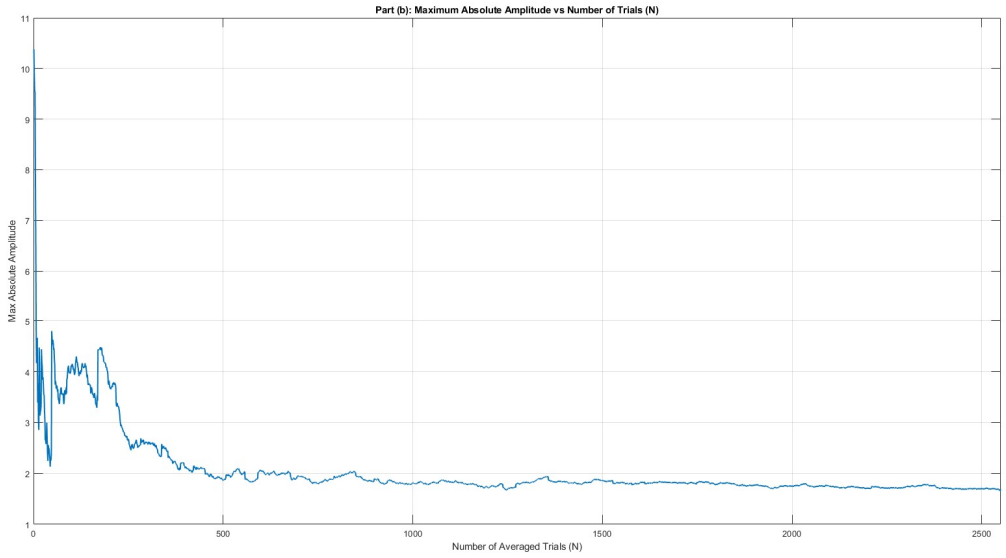
```
clear; clc; close all;
% Load the data
data=load("E:\6th Semester\MISP Lab\MyLab\LAB4\ERP_EEG.mat");
ERP_EEG1=data.ERP_EEG;
Fs = 240; % Sampling frequency (Hz)
t = (0:239) / Fs; % Time vector (0 to ~1 second)
total_trials = 2550;

%% Part (a): Average response for N = 100:100:2500

figure;
hold on;
colors = parula(25); % Colormap for 25 lines
c_idx = 1;

for N = 100:100:2500
    % Average across the first N columns (dimension 2)
    avg_response = mean(ERP_EEG1(:, 1:N), 2);

    plot(t, avg_response, 'Color', colors(c_idx,:), 'DisplayName', ['N = ', num2str(N)]);
    c_idx = c_idx + 1;
end
hold off;
title('Part (a): Average ERP for N = 100 to 2500');
xlabel('Time (s)');
ylabel('Amplitude');
% legend('show', 'Location', 'eastoutside'); % Optional: uncomment to see legend
grid on;
```



Part 2

Using the MATLAB script below, we plot the following results:

```
%% Part (b): Max absolute amplitude vs N (1 to 2550)
max_amp = zeros(1, total_trials);

for N = 1:total_trials
    % Average the first N trials
    avg_response = mean(ERP_EEG1(:, 1:N), 2);

    % Store the max absolute amplitude of the averaged ERP
    max_amp(N) = max(abs(avg_response));
end

figure;
plot(1:total_trials, max_amp, 'LineWidth', 1.5);
title('Part (b): Maximum Absolute Amplitude vs Number of Trials (N)');
xlabel('Number of Averaged Trials (N)');
ylabel('Max Absolute Amplitude');
grid on;
```

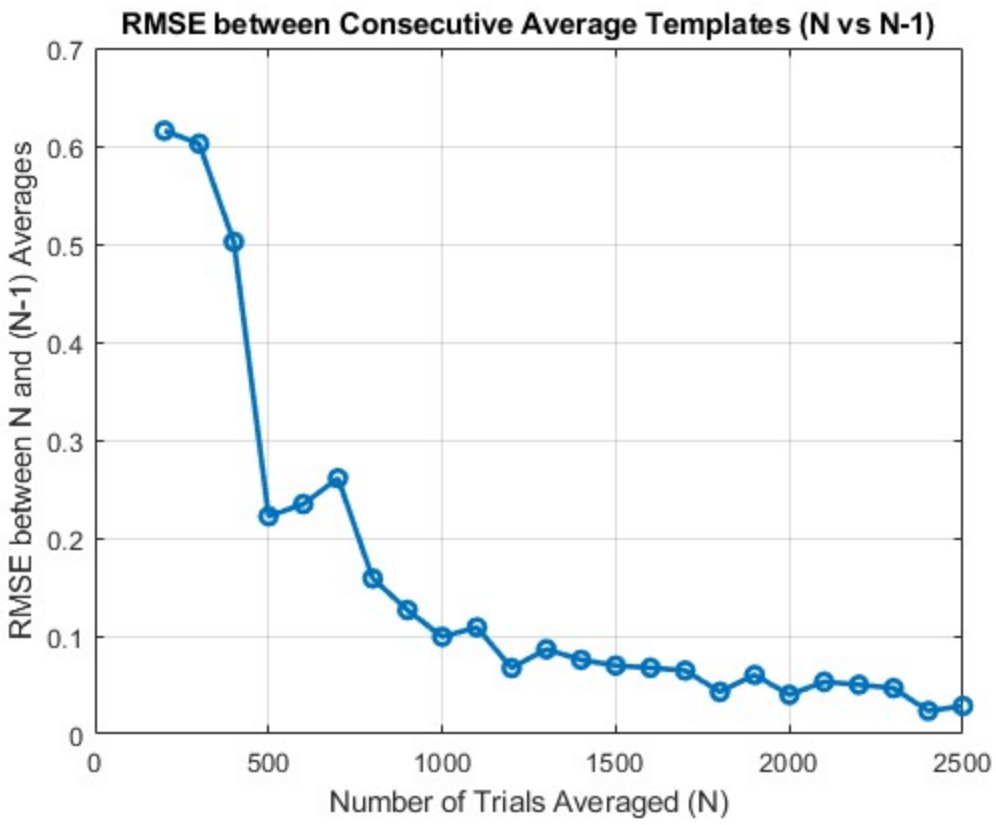


Beyond N = 500, the maximum absolute value remains unchanged, indicating that further averaging no longer alters the signal, and the extracted P300 has stabilized.

Part 3

Using the MATLAB script below, we plot the following results:

```
%% Part (c): Plot RMSE between i-th and (i-1)-th average templates

N_total_in_file = 2550;
n_values_for_rmse = 100:100:2500;
rmse_vals = zeros(1, length(n_values_for_rmse));

for i = 2:length(n_values_for_rmse)
    n = n_values_for_rmse(i);       % Current number of trials for averaging
    n_minus_1 = n_values_for_rmse(i-1); % Previous number of trials for averaging

    current_n_trials_data = ERP_EEG1(:, 1:n);
    previous_n_trials_data = ERP_EEG1(:, 1:n_minus_1);
    avg_erpn = mean(current_n_trials_data, 2);
    avg_erpn_minus_1 = mean(previous_n_trials_data, 2);

    % Calculate Root Mean Square Error (RMSE) between the two average templates.
    rmse_vals(i) = sqrt(mean((avg_erpn - avg_erpn_minus_1).^2));
end

figure;
plot(n_values_for_rmse(2:end), rmse_vals(2:end), '-o', 'LineWidth', 2, 'MarkerSize', 6);
xlabel('Number of Trials Averaged (N)');
ylabel('RMSE between N and (N-1) Averages');
title('RMSE between Consecutive Average Templates (N vs N-1)');
grid on;
```



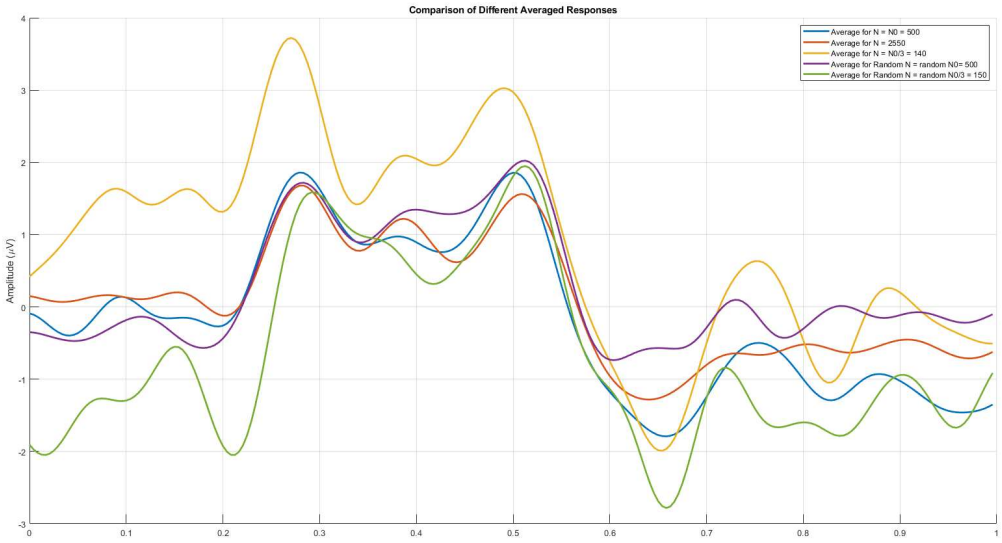
The RMS error comparing each successive average to its predecessor exhibits a nearly monotonic decreasing trend.

Part 4

Our results suggest that $N_0 = 500$ is a justifiable choice, since increasing N further results in negligible differences in the signal.

Part5

```
%% part(e)
N_0 = 500;
N_2550 = 2550;
N_0_3 = 150;

avg_erp_N_0 = mean(ERP_EEG1(:, 1:N_0), 2);
avg_erp_N_0_3 = mean(ERP_EEG1(:, 1:N_0_3), 2);
avg_erp_2550 = mean(ERP_EEG1, 2);

random_N_0_trials = ERP_EEG1(:, randperm(2550, N_0));
avg_erp_random_N_0_trials = mean(random_N_0_trials, 2);

random_N_0_3_trials = ERP_EEG1(:, randperm(2550, N_0_3));
avg_erp_random_N_0_3_trials = mean(random_N_0_3_trials, 2);

figure;
hold on;
plot(t, avg_erp_N_0, 'LineWidth', 2, 'DisplayName', 'Average for N = N0 = 500');
plot(t, avg_erp_2550, 'LineWidth', 2, 'DisplayName', 'Average for N = 2550');
plot(t, avg_erp_N_0_3, 'LineWidth', 2, 'DisplayName', 'Average for N = N0/3 = 140');

plot(t, avg_erp_random_N_0_trials, 'LineWidth', 2, 'DisplayName', 'Average for Random N = random N0= 500');
plot(t, avg_erp_random_N_0_3_trials, 'LineWidth', 2, 'DisplayName', 'Average for Random N = random N0/3 = 150');


ylabel('Amplitude (\muV)');
title('Comparison of Different Averaged Responses');
legend;
grid on;
```



As illustrated, the average signal obtained with $N_0 = 500$ captures the P300 information more accurately, as evidenced by the signal's fluctuations. In contrast, the other settings fail to reproduce the signal's complexity to the same extent.

Part6

In real-world P300-based BCIs—such as the widely-used P300 speller—the number of repetitions for each character typically ranges from **3 to 15**, depending on the specific system and desired performance. For example, some systems function with as few as 1 to 3 flashes, achieving meaningful accuracy in an online setting. A more typical configuration is **10 to 15 repetitions** per character, which balances speed and reliability. In some cases, especially in clinical or precision-focused applications, this can increase to around 15 to 20 sequences, but it rarely, if ever, approaches the **500 repetitions** our previous analysis indicated as an optimal value for signal stabilization.

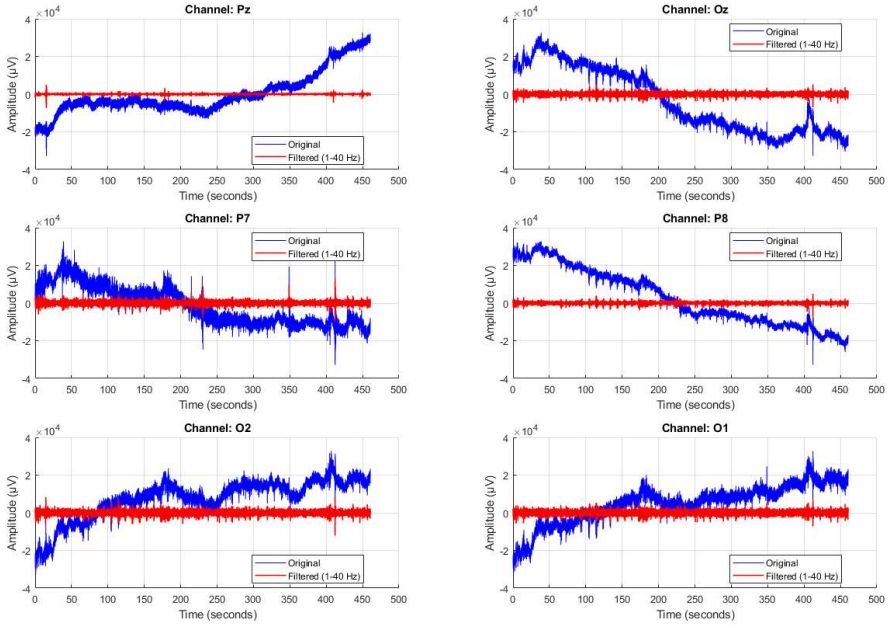
- Speed vs. SNR trade-off: Real BCIs prioritize speed. Using 600 trials would take minutes per selection, which is impractical.
- Advanced classifiers: BCIs use machine learning (e.g., SWLDA, CNNs) to detect P300 from single or few trials, not just averaging.
- Adaptive stopping: Systems stop flashing once confident, often needing only 5–15 repetitions on average.
- Goal difference: Your lab analysis maximized SNR for pure waveform extraction; real BCIs optimize accuracy vs. speed.

Section 2

Part1

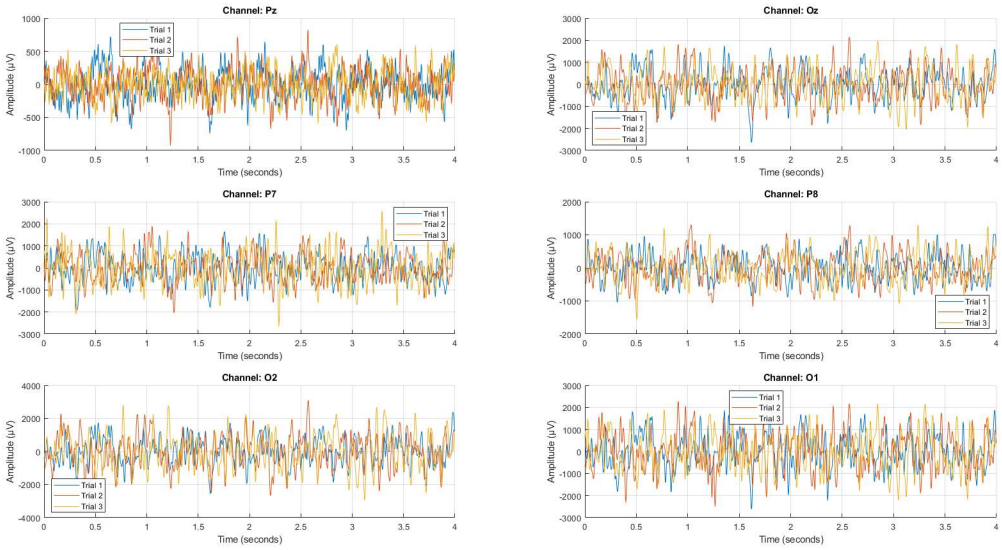
```
%% Load Data
load("E:\6th Semester\MISP Lab\MyLab\LAB4\SSVEP_EEG.mat");

% Extract data
eeg_data = SSVEP_Signal';
trigger_indices = Event_samples;
fs = 256;
num_channels = size(eeg_data, 2);
channel_names = {'Pz', 'Oz', 'P7', 'P8', 'O2', 'O1'};

fprintf('Data dimensions: %d samples × %d channels\n', size(eeg_data, 1), size(eeg_data, 2));
fprintf('Number of triggers: %d\n', length(trigger_indices));
```

```
%% Part (a): Bandpass Filtering (1-40 Hz) for ALL channels

% Design Butterworth bandpass filter
filter_order = 4;
low_cutoff = 1;
high_cutoff = 40;
[b, a] = butter(filter_order, [low_cutoff high_cutoff]/(fs/2), 'bandpass');

% Apply filter to ALL channels
filtered_data = zeros(size(SSVEP_Signal));
for ch = 1:num_channels
    filtered_data(ch, :) = filtfilt(b, a, SSVEP_Signal(ch, :));
end
figure('Name', 'Part (a): Original vs Filtered Signal - All Channels', 'Position', [50 50 1400 900]);
t = (0:size(SSVEP_Signal, 2)-1) / fs;

for ch = 1:num_channels
    subplot(3, 2, ch);
    hold on;
    plot(t, SSVEP_Signal(ch, :), 'b', 'LineWidth', 0.5, 'DisplayName', 'Original');
    plot(t, filtered_data(ch, :), 'r', 'LineWidth', 1.2, 'DisplayName', 'Filtered (1-40 Hz)');
    hold off;

    title(['Channel: ' channel_names{ch}]);
    xlabel('Time (seconds)');
    ylabel('Amplitude (μV)');
    legend('Location', 'best');
    grid on;
end
```



Part 2

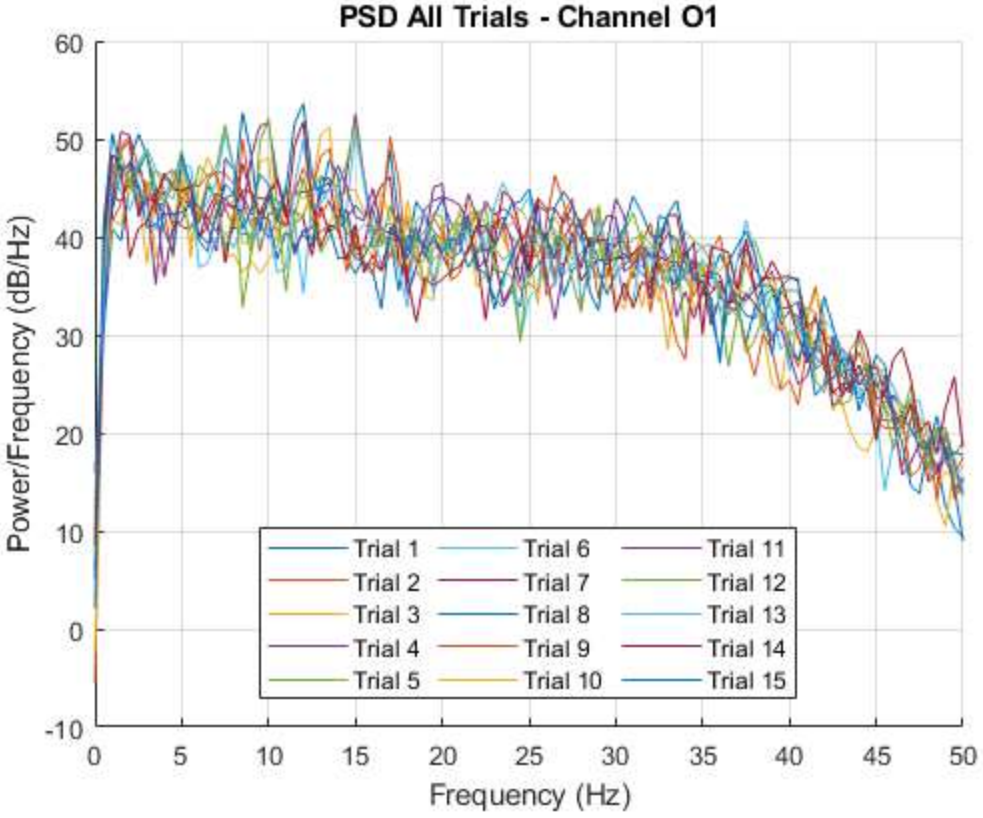
```
47  %% Part B) Extract trials (15 trials x 15 stimuli)
48  trial_length = 4 * fs;  % 4 seconds
49  num_trials = length(trigger_indices);
50  trials = zeros(trial_length, num_channels, num_trials);
51
52  for trial = 1:num_trials
53      start_idx = trigger_indices(trial);
54      end_idx = start_idx + trial_length - 1;
55
56      if end_idx <= size(filtered_data, 2)
57          trials(:, :, trial) = filtered_data(:, start_idx:end_idx)';
58      else
59          warning('Trial %d exceeds data length. Skipping.', trial);
60      end
61  end
62
63  fprintf('Extracted %d trials with length %d samples.\n', num_trials, trial_length);
64
65  % Plot example trials for all channels
66  figure('Name', 'Part (b): Example Trials - All Channels', 'Position', [50 50 1400 900]);
67  t_trial = (0:trial_length-1) / fs;
68
69  for ch = 1:num_channels
70      subplot(3, 2, ch);
71      hold on;
72      for trial = 1:min(3, num_trials)
73          plot(t_trial, trials(:, ch, trial), 'DisplayName', sprintf('Trial %d', trial));
74      end
75      hold off;
76
77      title(['Channel: ' channel_names{ch}]);
78      xlabel('Time (seconds)');
79      ylabel('Amplitude (μV)');
80      legend('Location', 'best');
81      grid on;
82  end
83
84
```

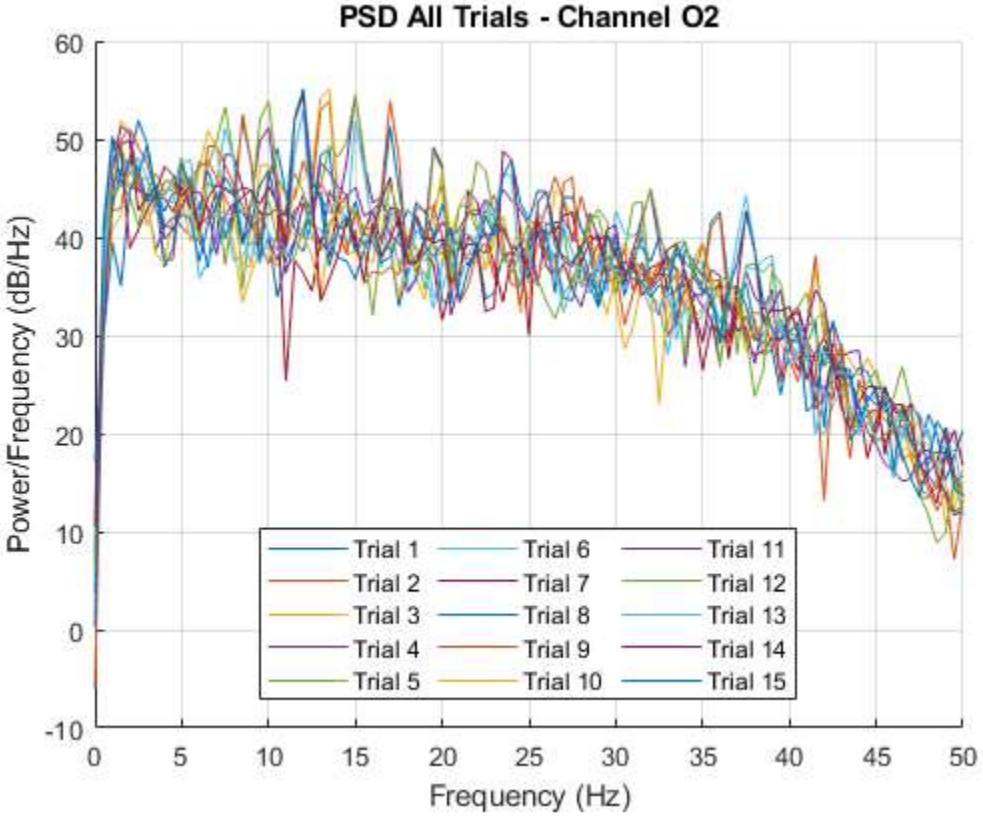


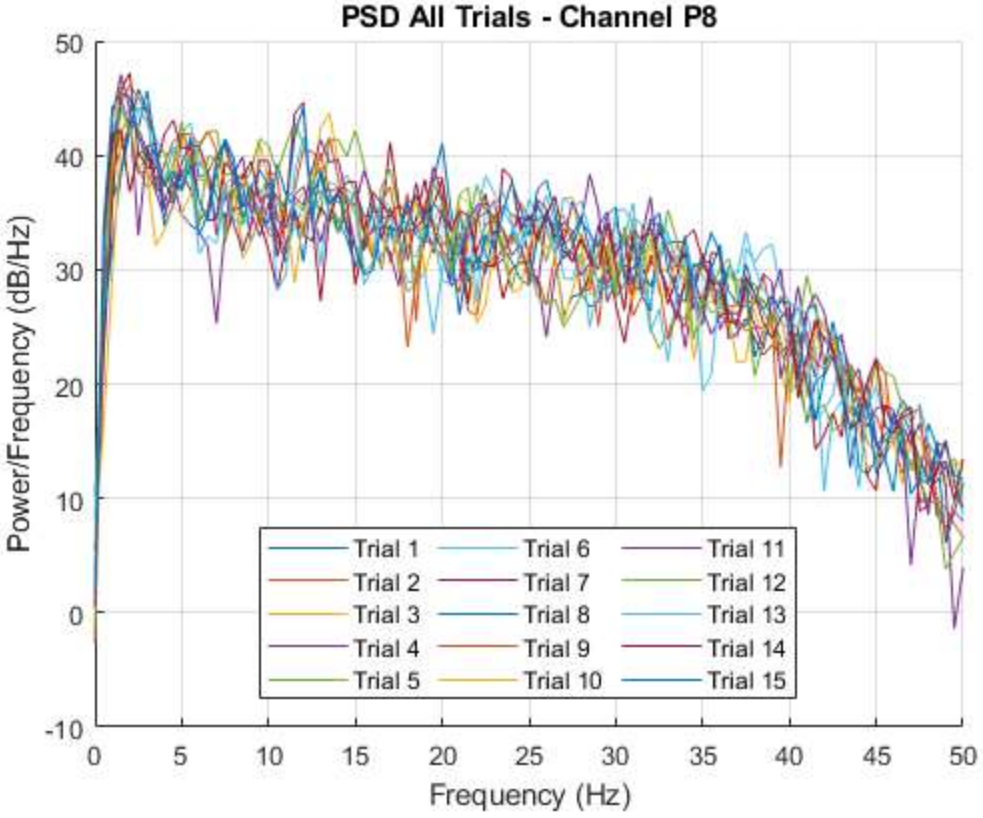
Part3

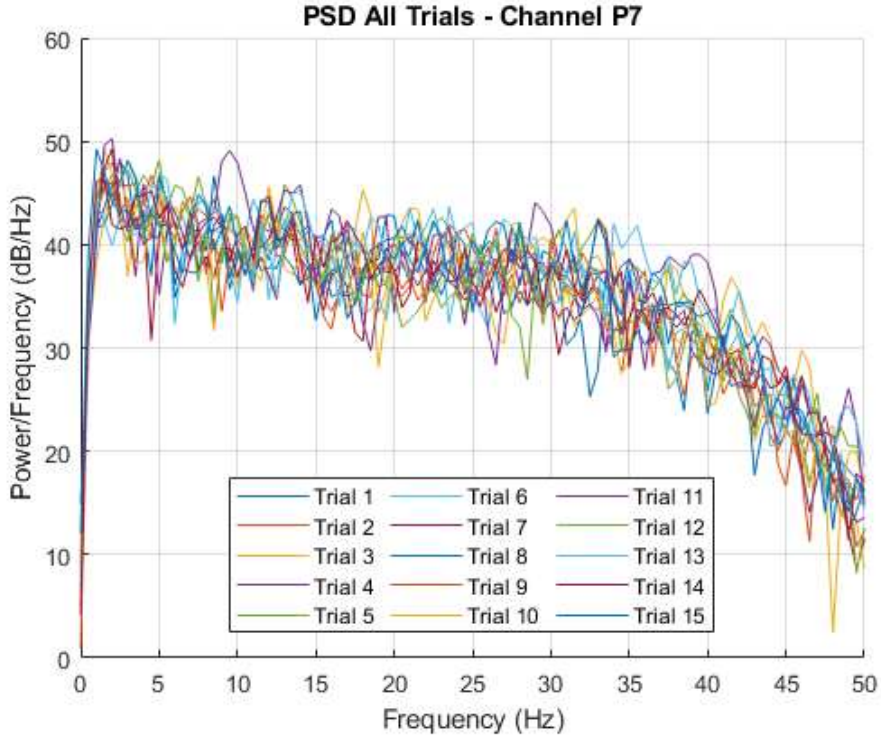
All 15 trials are shown in the same plot for each channel.

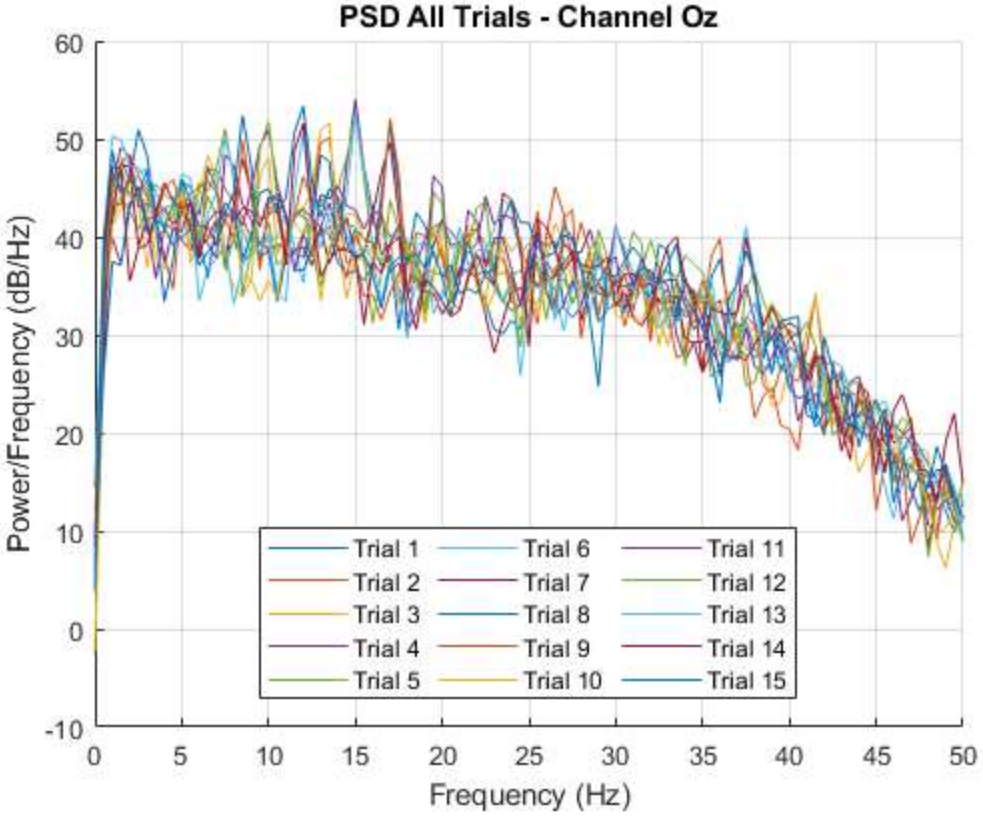
```
%% Part C) Calculate and plot PSD for each trial
pwelch_window = 2 * fs;
overlap = pwelch_window / 2;
nfft = 2^nextpow2(pwelch_window);

% Calculate PSD for each trial and each channel
psd_all = cell(num_trials, num_channels);
freq_all = [];

for trial = 1:num_trials
    for ch = 1:num_channels
        [psd, freq] = pwelch(trials(:, ch, trial), pwelch_window, overlap, nfft, fs);
        psd_all{trial, ch} = psd;
        if isempty(freq_all)
            freq_all = freq;
        end
    end
end

fprintf('PSD calculation completed.\n');

% Plot PSD for all trials - each channel in a separate figure
for ch = 1:num_channels
    figure('Name', sprintf('PSD All Trials - %s', channel_names{ch}));
    hold on;
    colors = lines(num_trials);

    for trial = 1:num_trials
        plot(freq_all, 10*log10(psd_all{trial, ch}), 'Color', colors(trial,:), ...
             'DisplayName', sprintf('Trial %d', trial));
    end

    xlabel('Frequency (Hz)');
    ylabel('Power/Frequency (dB/Hz)');
    title(sprintf('PSD All Trials - Channel %s', channel_names{ch}));
    xlim([0 50]);
    grid on;
    legend('Location', 'best', 'NumColumns', 3);
    hold off;
end

```

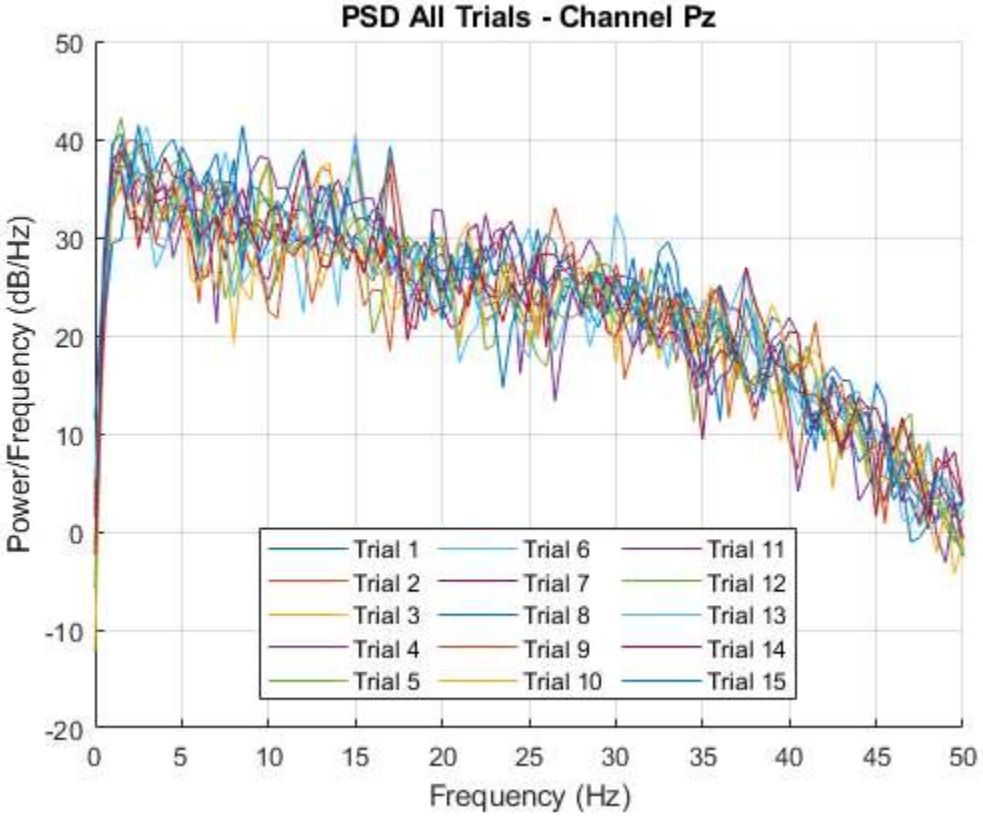

PSD All Trials - Channel O1
Power/Frequency (dB/Hz)
Frequency (Hz)
Trial 1
Trial 2
Trial 3
Trial 4
Trial 5
Trial 6
Trial 7
Trial 8
Trial 9
Trial 10
Trial 11
Trial 12
Trial 13
Trial 14
Trial 15


PSD All Trials - Channel O2
Power/Frequency (dB/Hz)
Frequency (Hz)
Trial 1
Trial 2
Trial 3
Trial 4
Trial 5
Trial 6
Trial 7
Trial 8
Trial 9
Trial 10
Trial 11
Trial 12
Trial 13
Trial 14
Trial 15

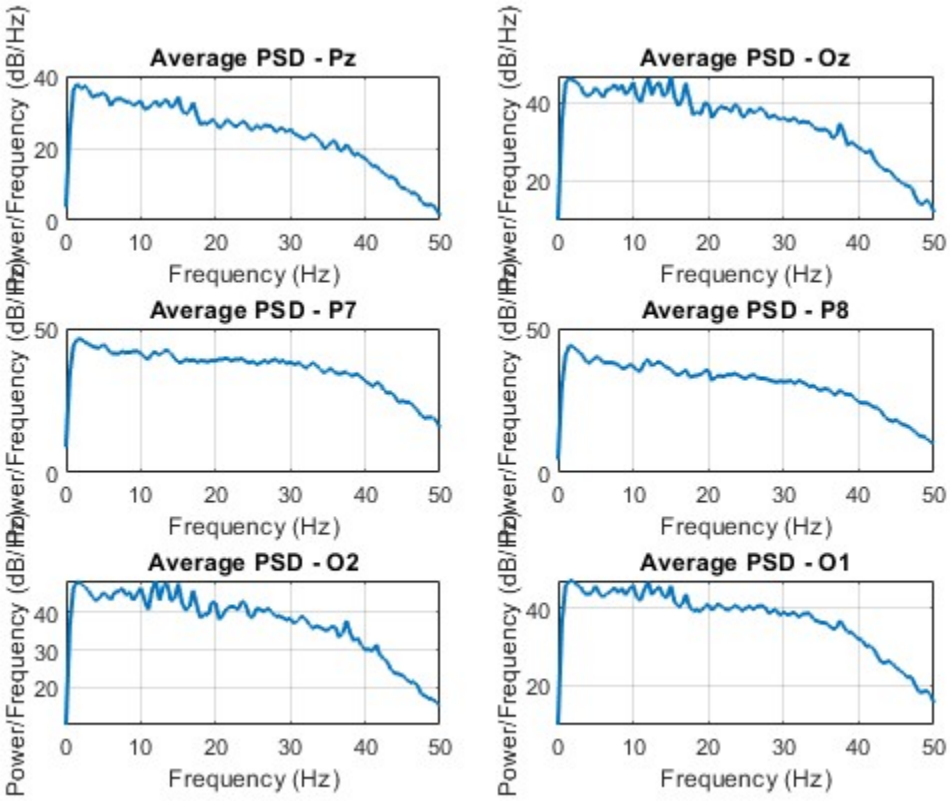

PSD All Trials - Channel P8
Power/Frequency (dB/Hz)
Frequency (Hz)
Trial 1
Trial 2
Trial 3
Trial 4
Trial 5
Trial 6
Trial 7
Trial 8
Trial 9
Trial 10
Trial 11
Trial 12
Trial 13
Trial 14
Trial 15


PSD All Trials - Channel P7
Power/Frequency (dB/Hz)
Frequency (Hz)
Trial 1
Trial 2
Trial 3
Trial 4
Trial 5
Trial 6
Trial 7
Trial 8
Trial 9
Trial 10
Trial 11
Trial 12
Trial 13
Trial 14
Trial 15


PSD All Trials - Channel Oz
Power/Frequency (dB/Hz)
Frequency (Hz)
Trial 1
Trial 2
Trial 3
Trial 4
Trial 5
Trial 6
Trial 7
Trial 8
Trial 9
Trial 10
Trial 11
Trial 12
Trial 13
Trial 14
Trial 15


PSD All Trials - Channel Pz
Power/Frequency (dB/Hz)
Frequency (Hz)
Trial 1
Trial 2
Trial 3
Trial 4
Trial 5
Trial 6
Trial 7
Trial 8
Trial 9
Trial 10
Trial 11
Trial 12
Trial 13
Trial 14
Trial 15

```
%% Average PSD across all trials
avg_psd = zeros(length(freq_all), num_channels);
for ch = 1:num_channels
    psd_matrix = cell2mat(cellfun(@(x) x, psd_all(:, ch), 'UniformOutput', false)');
    avg_psd(:, ch) = mean(psd_matrix, 2);
end

% Plot average PSD for all channels
figure('Name', 'Average PSD - All Channels');
num_rows = ceil(num_channels / 2);
for ch = 1:num_channels
    subplot(num_rows, 2, ch);
    plot(freq_all, 10*log10(avg_psd(:, ch)), 'LineWidth', 1.5);
    xlabel('Frequency (Hz)');
    ylabel('Power/Frequency (dB/Hz)');
    title(sprintf('Average PSD - %s', channel_names{ch}));
    xlim([0 50]);
    grid on;
end

fprintf('Analysis completed.\n');
```



Part4

No, the frequency response differs from trial to trial. This variability arises because the brain's response is not deterministic; it is modulated by internal states like attention, alertness, and habituation, as well as ongoing background EEG activity. Therefore, the spectral characteristics of the signal are not identical across repeated trials.

Part5

Within each trial, every channel exhibits a dominant frequency component that stands out more significantly than others. The highest peak in the frequency spectrum corresponds to the fundamental stimulation frequency, confirming that the brain's steady-state visual evoked response locks to the driving frequency. Additionally, clear peaks are also observed at integer multiples (harmonics) of this fundamental frequency, which arise due to the non-linear nature of the visual system's response to periodic stimulation.

Part6

While analyzing the frequency content of each individual channel can reveal the stimulation frequency, more robust and accurate methods exist. These techniques typically exploit information across multiple channels simultaneously, leading to improved signal-to-noise ratio and higher classification accuracy.

The most widely adopted method is Canonical Correlation Analysis (CCA), which finds the maximal correlation between the multi-channel EEG data and a set of reference sine-cosine templates at each candidate frequency and its harmonics. A more advanced variant, Filter Bank CCA (FBCCA), further improves performance by decomposing the signal into multiple sub-bands and weighting their contributions.

Another powerful approach is Task-Related Component Analysis (TRCA), which enhances the reproducibility of task-related components across trials, making it particularly effective for SSVEP detection. Multivariate Synchronization Index (MSI) is also used to quantify the synchronization between the recorded EEG and reference signals.

In recent years, deep learning-based methods such as EEGNet, FB-MultiCNN, and SSVEP-TFFNet have demonstrated state-of-the-art results by automatically learning optimal spatial-temporal features from raw multi-channel data without manual feature engineering.

These methods consistently outperform single-channel spectral analysis, especially in low-SNR conditions or when the stimulation frequency is close to background EEG rhythms.

Lab 2: Medical Image and Signal Processing Laboratory

Sharif University of Technology

Spring 1405

Negin Raki
neginnima01@gmail.com

Amitis Mirabedini
mirabediniamitis@gmail.com

Part3:ERS and ERD analyzing

The goal of this section is to analyze Event-Related Synchronization/Desynchronization (ERS/ERD) during 5 different mental tasks (word generation, mental subtraction, spatial navigation, right hand motor imagery, and feet motor imagery) using 30-channel EEG data.

Part (a): Frequency Band Filtering

Explanation

Raw EEG signals are a complex mix of different brain frequencies. To analyze specific cognitive functions, you need to decompose the original signal into well-known physiological frequency bands. We are asked to apply bandpass filters to the raw EEG data to extract four distinct bands:

- **Delta (δ):** 1–4 Hz
- **Theta (θ):** 4–8 Hz
- **Alpha (α):** 8–13 Hz
- **Beta (β):** 13–30 Hz

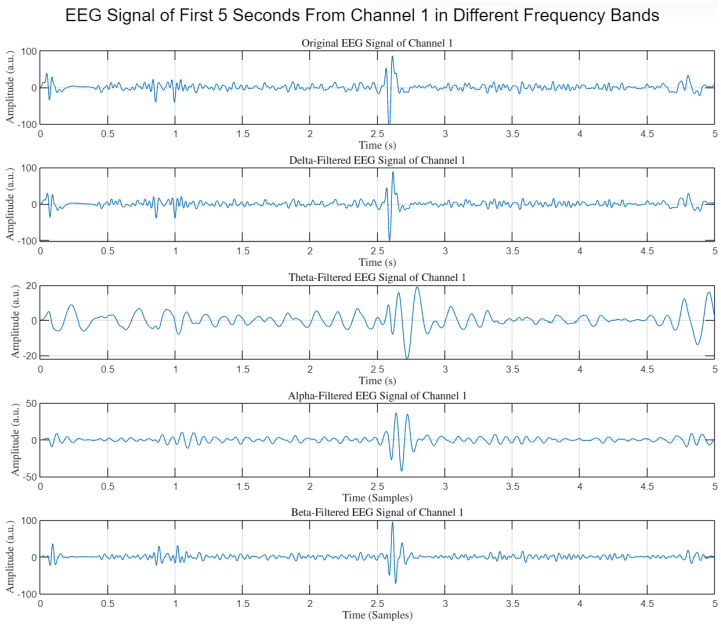


Figure 1: EEG Signal of First 5 Seconds From Channel 1 in Different Frequency Bands

The generated plot visualizes this filtering step. It displays the original raw EEG signal alongside the signals after being successfully filtered into the Delta, Theta, Alpha, and Beta bands for the first 5 seconds of Channel 1.

Part (b): Trial Extraction (Epoching)

Explanation

The EEG data is recorded as a long, continuous stream. However, the experiment consists of multiple distinct tasks (trials). To analyze the brain's reaction to each specific task, We must segment (epoch) this continuous data into smaller, discrete chunks.

```
%% Part 3.2
for i=1:200
   Delta_Trials(:,:,i) = Delta_X(trial(i): trial(i)+256*10,:);
   Theta_Trials(:,:,i) = Theta_X(trial(i): trial(i)+256*10,:);
   Alpha_Trials(:,:,i) = Alpha_X(trial(i): trial(i)+256*10,:);
   Beta_Trials(:,:,i) = Beta_X(trial(i): trial(i)+256*10,:);
end
```

Part (c): Power Calculation

Task: Calculate the power of each data point by squaring the amplitude of all time points for each trial, in every frequency band and channel.
Outcome: Squaring the amplitude transforms the bipolar oscillating signal into a strictly positive power representation. This highlights the energy level of the EEG signal at any given time, making amplitude modulations obvious.

```
%% Part 3.3
for i = 1:200
    for j = 1:30
        Power_Delta(:,j,i) = (Delta_Trials(:,j,i).^2);
        Power_Theta(:,j,i) = (Theta_Trials(:,j,i).^2);
        Power_Alpha(:,j,i) = (Alpha_Trials(:,j,i).^2);
        Power_Beta(:,j,i) = (Beta_Trials(:,j,i).^2);
    end
end
```

Part (d): Class-based Averaging

Task: Separate the data into 5 classes based on the trial labels (from the 200 total trials) and calculate the average power for each channel across the trials of the same class. This yields four tensors: `Delta_X_avg`, `Theta_X_avg`, `Alpha_X_avg`, and `Beta_X_avg`.
Outcome: Single-trial EEG data contains a high level of background noise. Averaging across trials of the same class suppresses this random noise and preserves the task-specific Event-Related Synchronization (ERS) or Desynchronization (ERD) patterns.

```
for class = 1:5
    Delta_X_Avg(:,:,class) = mean(Power_Delta(:,:,(y==class)),3);
    Theta_X_Avg(:,:,class) = mean(Power_Theta(:,:,(y==class)),3);
    Alpha_X_Avg(:,:,class) = mean(Power_Alpha(:,:,(y==class)),3);
    Beta_X_Avg(:,:,class) = mean(Power_Beta(:,:,(y==class)),3);
end
```

Part (e): Signal Smoothing

Task: Smooth the temporal variations of the averaged signals from Part (d) using a rectangular window and the convolution operation (`conv`).
Outcome: The output will be a smooth envelope of the power signal. This moving average removes high-frequency fluctuations, making the macro-trends of the ERS/ERD over the 10-second window visually discernible.

```
%% Part 3.5
newWin = ones(1,200) / sqrt(200);
for class = 1:5
    for ch = 1:30
        Delta_Avg_Smooth(:,ch,class) = conv(Delta_X_Avg(:,ch, class), newWin, "same");
        Theta_Avg_Smooth(:,ch,class) = conv(Theta_X_Avg(:,ch, class), newWin, "same");
        Alpha_Avg_Smooth(:,ch,class) = conv(Alpha_X_Avg(:,ch, class), newWin, "same");
        Beta_Avg_Smooth(:,ch,class) = conv(Beta_X_Avg(:,ch, class), newWin, "same");
    end
end
```

Part (f): Plotting the Average Signal

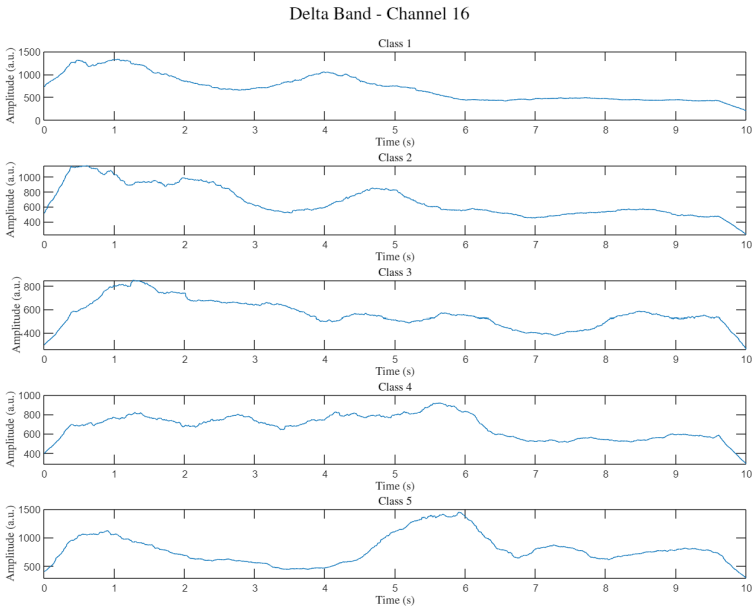
Task: Select a central channel (e.g., `PC`) and plot the 10-second smoothed temporal average signals for the 5 classes in each frequency band.
Outcome: Running this code will generate 4 separate figures (one for δ, θ, α, and β bands). Each figure will contain 5 overlaid plots corresponding to the 5 mental tasks, allowing for visual comparison of how different tasks affect different frequency bands.

```
%% Part 3.6
% CPz channel = 16
band_data = {
    Delta_Avg_Smooth, 'Delta';
    Theta_Avg_Smooth, 'Theta';
    Alpha_Avg_Smooth, 'Alpha';
    Beta_Avg_Smooth,  'Beta';
};
t = 10;
n_samples = size(Delta_Avg_Smooth, 1);   % = 2561
duration = linspace(0, t, n_samples);    % time vector in seconds
% Loop over each band
for b = 1:size(band_data,1)
    data = band_data{b,1};
    band_name = band_data{b,2};

    figure('Position',[0,0,1000,750]);
    for cls = 1:5
        subplot(5,1,cls);
        plot(duration,data(:,16,cls));  % 16 = channel index
        %xline(768, Color='r', LineWidth=1.5);
        xlabel('Time (s)', 'Interpreter','latex');
        ylabel('Amplitude (a.u.)', 'Interpreter','latex');
        title(['Class ' num2str(cls)], 'Interpreter','latex');
    end

    sgtitle([band_name ' Band - Channel 16'], 'Interpreter','latex');
end
```

Part (g): Conclusion and Analysis

1. Delta Band Analysis

Delta rhythms are typically slow waves, and changes in this band can be related to task engagement or attention focusing.



- **Class 1:** The power reaches a high peak during the baseline rest period (around $t = 1$). After the cue at $t = 3$, the power generally decreases, with a minor bump near $t = 4$, showing an overall ERD trend throughout the task execution.
- **Class 2:** Similar to Class 1, there is a sharp baseline peak early on. Following $t = 3$, the power drops, exhibits a small rebound between $t = 4.5$ and $t = 5.5$, and then continues to decrease (overall ERD).
- **Class 3:** Displays a broad peak during the baseline phase. After the task begins at $t = 3$, the signal shows a steady, gradual decline, representing a continuous ERD.
- **Class 4:** The baseline power is relatively stable. After $t = 3$, there is a slight dip followed by a noticeable, sustained increase in power (ERS) that peaks around $t = 5.5$ to $t = 6$ seconds, before dropping off towards the end of the task.
- **Class 5:** Features a prominent baseline peak that dips right before the task cue. After $t = 3$, the power drops further but is followed by a massive, sharp spike (strong ERS) peaking just before $t = 6$.

2. Alpha Band Analysis

In general, Alpha rhythms often desynchronize (decrease in power) during active cognitive or motor tasks.

- **Class 1:** The baseline power is quite high. Immediately after the cue at $t = 3$, there is a distinct power drop (ERD) that remains low for the duration of the task, with a slight recovery towards the end.
- **Class 2:** Similar to Class 1, there is a relatively high baseline. After $t = 3$, the power noticeably decreases and stays suppressed (ERD) throughout the task period.

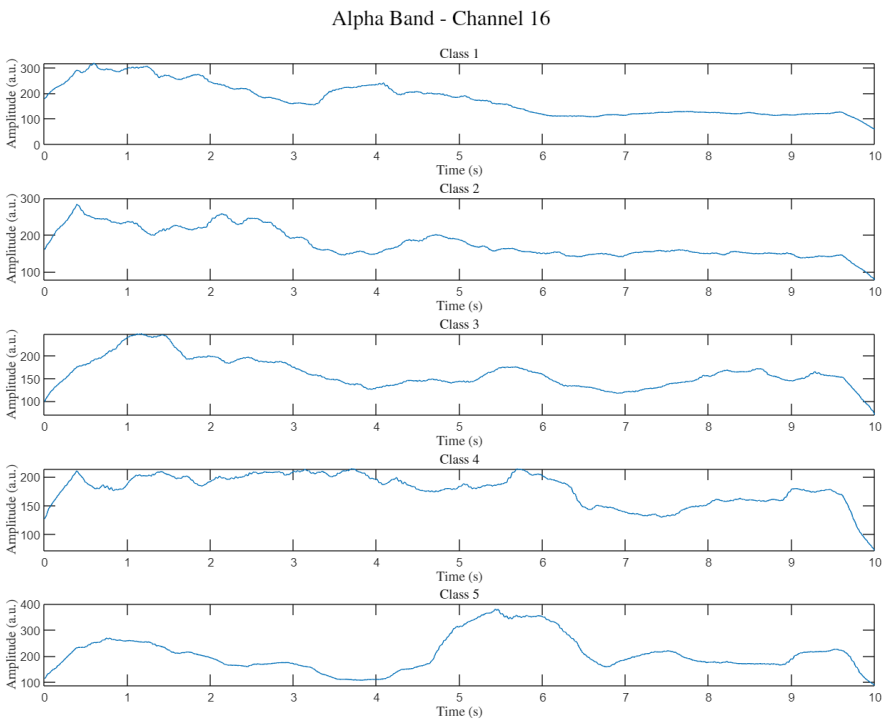


Figure 2: Alpha band power envelope for Channel 16 across 5 classes.

- **Class 3:** The signal shows a strong peak during the baseline rest period. After the $t = 3$ cue, the power drops significantly (ERD) and remains lower than the baseline peak for the rest of the trial.
- **Class 4:** The baseline power is high and stable. Interestingly, the power remains high even after the task cue at $t = 3$, indicating a lack of immediate ERD. It only begins to drop significantly around $t = 6$.
- **Class 5:** This class shows a very distinct pattern. The power drops initially during the task, but then exhibits a massive surge in power (strong ERS) between $t = 4.5$ and $t = 6.5$, before dropping back down.

3. Beta Band Analysis

Beta rhythms are often associated with active thinking, focus, or motor activity.

- **Class 1:** The baseline power starts very high and gradually trends downward. This downward trend (ERD) continues steadily throughout the task execution phase after $t = 3$.
- **Class 2:** Exhibits a pattern very similar to Class 1. High initial baseline power that steadily decreases (ERD) over the course of the 10-second window, seemingly unaffected by the $t = 3$ cue.

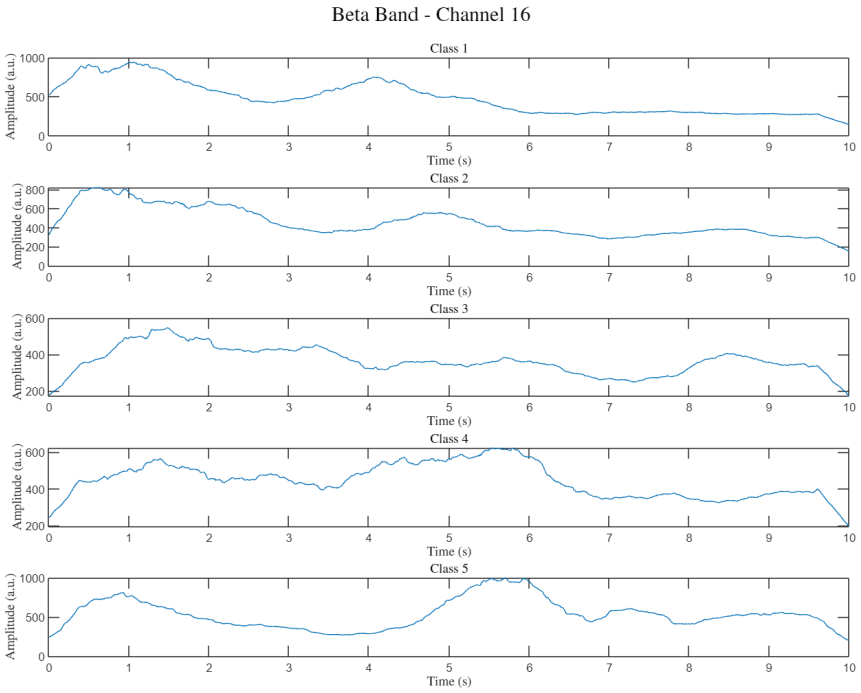


Figure 3: Beta band power envelope for Channel 16 across 5 classes.

- **Class 3:** The baseline power fluctuates but remains relatively moderate. After $t = 3$, the power drops slightly and remains fairly flat and suppressed (mild ERD) for most of the task duration.
- **Class 4:** The baseline shows some fluctuation. However, after the task begins at $t = 3$, there is a clear and sustained increase in power (ERS) that peaks around $t = 5.5$ to $t = 6$ before tapering off.
- **Class 5:** The baseline power dips right before the cue. After $t = 3$, similar to Class 4 but even more pronounced, there is a massive spike in beta power (strong ERS) peaking around $t = 5.5$ to $t = 6$.

Summary Observation

A highly consistent pattern emerges across the Delta, Alpha, and Beta bands. Classes 4 and 5 distinctively show Event-Related Synchronization (power increases) peaking around $t = 5.5$ to $t = 6$ seconds during the task. Class 5's ERS is particularly explosive across all three analyzed bands. Conversely, Classes 1, 2, and 3 generally exhibit Event-Related Desynchronization (power decreases) across the bands once the tasks begin.